OpenClaw Vector Memory

OpenClaw Vector Memory

1. Course Content
    Course Overview
        What is Vector Memory
        How It Works
        Use Cases
        Workflow
        Features
2. Prerequisites
3. Memory Feature Demo Case
4. Retrieving Memory via Vector Search
    4.1 Prerequisites
    4.2 Configure embeddings Model API_KEY
    4.3 Verify Vector Memory

1. Course Content

Course Overview

OpenClaw has powerful memory capabilities that enable the robot to remember important information during conversations and intelligently retrieve and use these memories in subsequent interactions. This section will introduce OpenClaw's Vector Memory (Semantic Memory) system, which is an intelligent memory retrieval mechanism based on semantic understanding.

What is Vector Memory

Vector Memory is OpenClaw's built-in **semantic search memory system**, which implements intelligent memory retrieval through the following core technologies:

- **Vector Embedding**: Converts memory documents into high-dimensional vector representations, enabling computers to understand the semantic meaning of text
- **Semantic Retrieval**: Matches based on semantic similarity of queries, not just keyword matching, can understand synonyms and related concepts

How It Works

1. **Vectorization Processing**
    - Converts content from MEMORY.md and memory/*.md into vector representations
    - Supports semantic understanding, not just keyword matching
2. **Semantic Retrieval**
    - Based on the semantic meaning of queries, not exact keywords
    - Returns the most relevant text fragments and their location information

Use Cases

Scenario	Example
Recalling Past Decisions	"The decision about sorting order before"
Finding Related Experience	"How to handle robotic arm grasping failure"
Task Context	"The project the user mentioned last week"
Personal Preferences	"User's preferred interaction method"

Workflow

```
User Question → memory_search(Semantic Query) → Returns Relevant Fragments → memory_get(Read Specific Location) → Generate Response
```

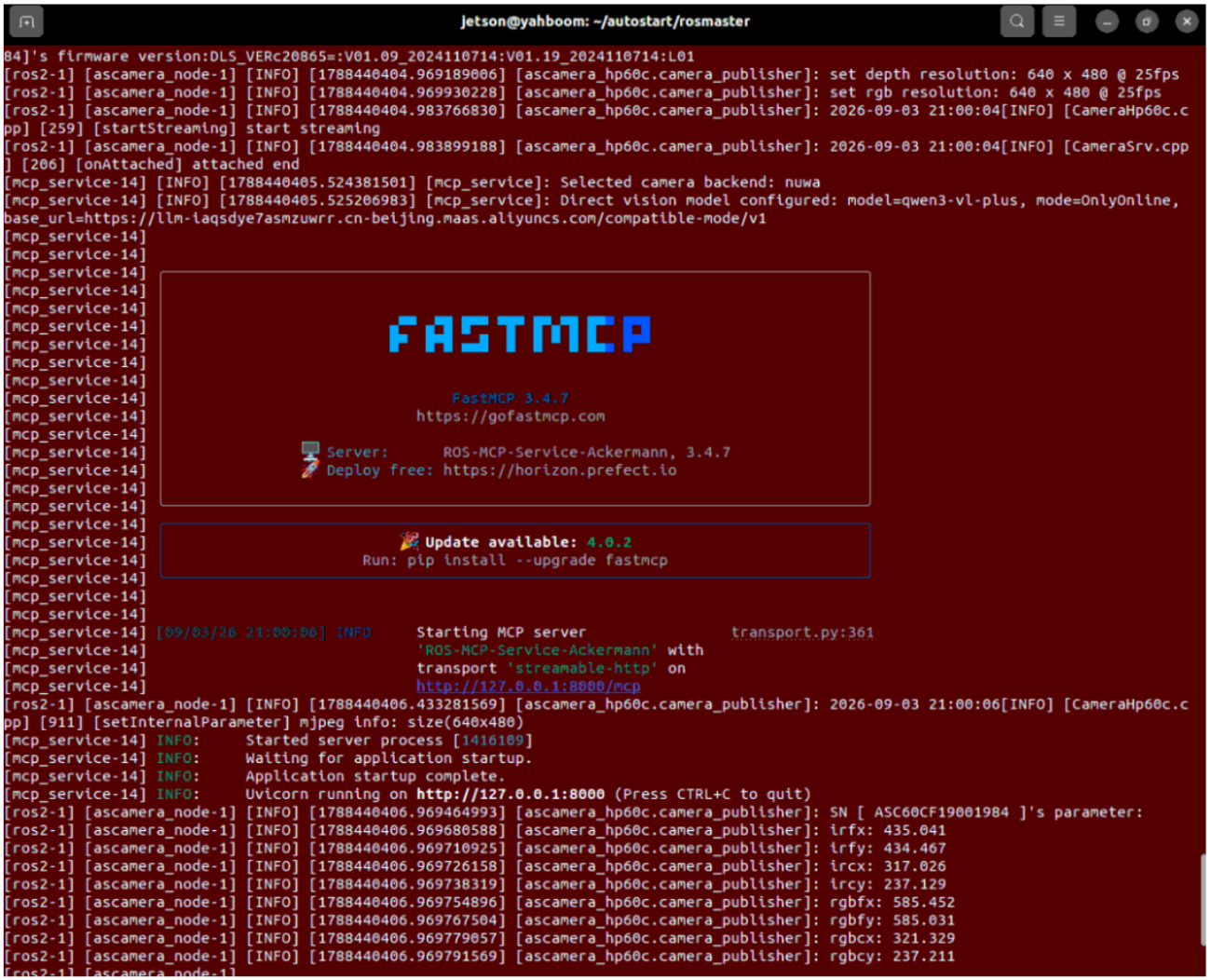
Features

- **Intelligent Matching**: Understands synonyms, related concepts
- **Automatic Citation**: Results include file path and line number
- **Complementary to File Memory**: Vector memory for retrieval, file memory for persistent storage

2. Prerequisites

- Complete the 【OpenClaw Skills Development】 chapter first, and master creating new skills

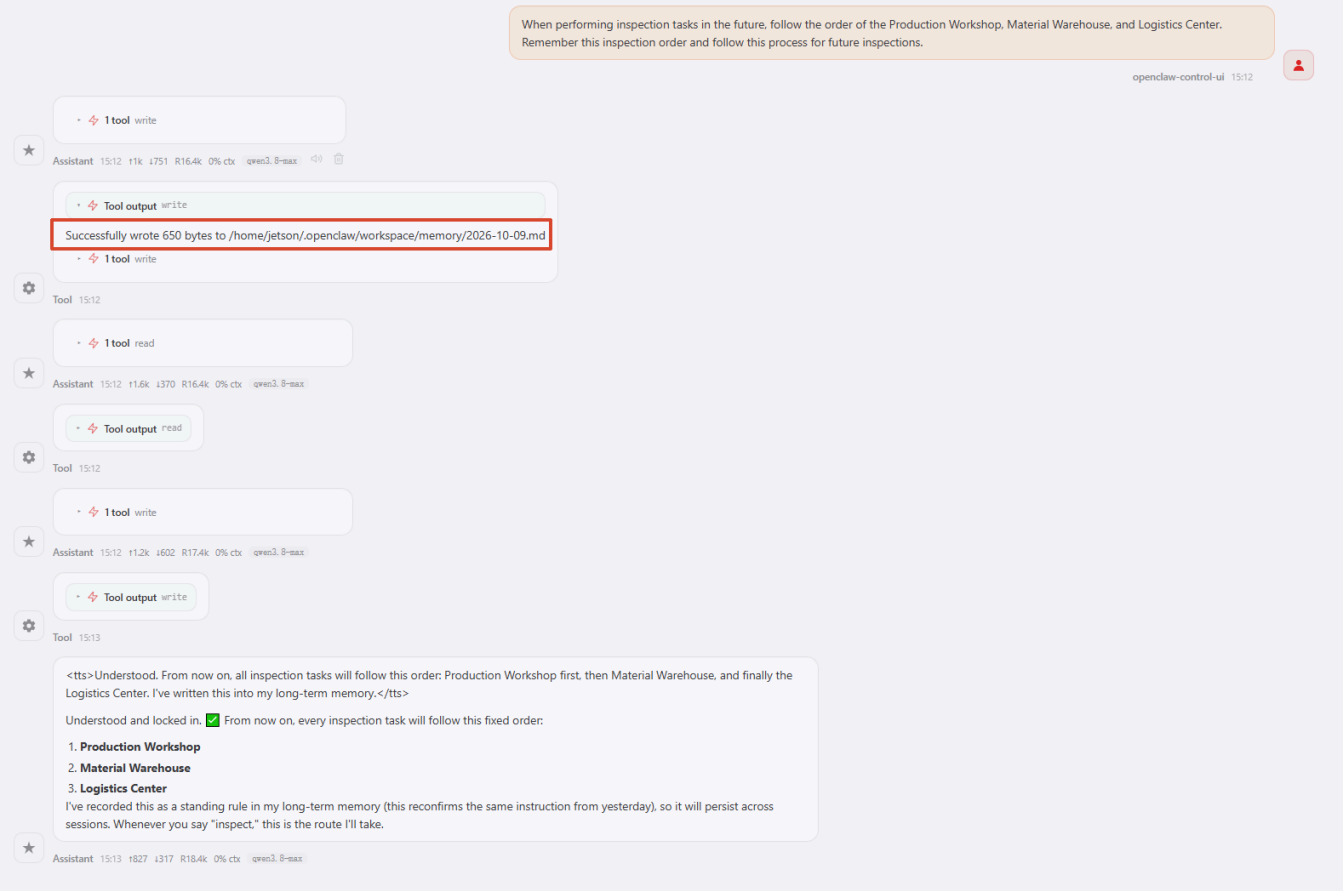
```
ros2 launch dofbot_pro_bringup mcp_service.launch.py
```



3. Memory Feature Demo Case

> **Tip**
>
> - This case is for demonstration reference only, you can have free conversations based on your own needs. The following is a case demonstration using WebChat as an example

- When performing inspection tasks in the future, follow the order of **the Production Workshop, Material Warehouse, and Logistics Center**. Remember this inspection order and follow this process for future inspections.



- OpenClaw's memory content is stored in files at the following path. You can view the memory content from the above case based on the actual generated name:

```
cat ~/.openclaw/workspace/memory/2026-09-03.md
# For Raspberry Pi version:
cat ~/MODELS/openclaw/workspace/memory/2026-09-03.md
```

```
jetson@yahboom:~$ cat ~/.openclaw/workspace/memory/2026-10-09.md
# 2026-10-09 — Daily Log

## Inspection Order Reconfirmed (15:12, openclaw-control-ui)

- User reconfirmed the standing directive: for ALL future inspection tasks, follow this fixed order:
  1. **Production Workshop** (生产车间)
  2. **Material Warehouse** (物料仓库)
  3. **Logistics Center** (物流中心)
- Originally given 2026-10-08; already in MEMORY.md. Reconfirmed today — treat as firm standing rule.
- Map symbols (as of 2026-10-08): 测试区1=A, 测试区2=B, FoodWaste=D, OtherWaste=C, 停车区=E, 1号货架=F, 产线=G, 2号货架=H, 装卸区=I, 维修区=J.
- Area→symbol mapping still tentative (Workshop≈产线G, Warehouse≈1号/2号货架F/H, Logistics≈装卸区I); confirm with user on first inspection run.
```

4. Retrieving Memory via Vector Search

- OpenClaw supports slicing and retrieving memory documents in vector mode, suitable for semantic retrieval of memories in large-scale memory documents

> **Tip**
>
> - OpenClaw's built-in default mode is **full-text search mode**, suitable for the vast majority of use cases. If you have higher requirements for the memory feature, you can refer to the following tutorial to implement vector storage of memory documents by adding an external **embeddings** model

4.1 Prerequisites

- You need to register an account with an **embeddings** model provider. The following demonstration uses the **openrouter** platform as an example. For account registration and API application, refer to the tutorial 【AI Model Development—Register Model Service Account—3. openrouter Platform Account】

4.2 Configure embeddings Model API_KEY

- Set the model provider (openai interface protocol)

```
openclaw config set agents.defaults.memorySearch.provider "openai"
```

- Set the baseUrl, here using openrouter as an example. If you are using **embeddings** models from other providers, replace with the actual baseUrl

```
openclaw config set agents.defaults.memorySearch.remote.baseUrl
"https://openrouter.ai/api/v1"
```

- Set the embeddings model, here using `"openai/text-embedding-3-small"` model as an example. If you are using other **embeddings** models, replace with the actual name

```
openclaw config set agents.defaults.memorySearch.model "openai/text-embedding-3-small"
```

- Set the API_KEY for the **embeddings** model

```
openclaw config set agents.defaults.memorySearch.remote.apiKey "sk-or-v1-
133949f96bec556343539a587f53d077xxxxxxxxxxxxxxxx"
```

- Restart the OpenClaw gateway to take effect

```
openclaw gateway restart
# For Raspberry Pi version:
ocg restart
```

4.3 Verify Vector Memory

- Check if vector memory is configured and active. The following output proves the configuration is successful, where Vector is the path of the sliced file

```
openclaw memory status --deep
```

```
Memory Search (main)
Provider: openai (requested: openai)
Model: text-embedding-3-small
Sources: memory
Indexed: 0/1 files · 0 chunks
Dirty: yes
Store: ~/.openclaw/memory/main.sqlite
Workspace: ~/.openclaw/workspace
Embeddings: ready
By source:
  memory · 0/1 files · 0 chunks
Vector: ready
Vector path: /usr/lib/node_modules/openclaw/node_modules/sqlite-vec-linux-arm64/vec0.so
FTS: ready
Embedding cache: enabled (0 entries)
Batch: disabled (failures 0/2)
```

- After initial configuration, you can manually update the index once

```
openclaw memory index --force
```

```
Memory index updated (main).
```

- Test if memory retrieval can find the memory fragments from the demonstration case above

```
openclaw memory search "inspection order"
```

- Here `0.362` is the semantic relevance score. A higher score indicates higher semantic similarity. Results greater than 0.3 typically have good relevance

